Name : Tanmay Mahesh Ahuja	**Semester :** V	**Branch :** AI&DS
Roll No : 03	**Batch :** T11	**Date :** 07-08-2026

Experiment No. 04

Aim : To set up a Continuous Integration pipeline using Jenkins Actions.

LO Mapped : LO3

LO : To design and configure Continuous Integration / Continuous Deployment pipelines using Jenkins, GitHub Actions and containerization tools.

Theory

Continuous Integration (CI) Architecture

Continuous Integration (CI) is a foundational DevOps development practice wherein developers merge code changes frequently into a central repository. Rather than delaying integration until the end of a sprint or development cycle, CI promotes automated builds and execution of automated testing suites every time code is committed.



Core benefits of Continuous Integration include:

- **Early Defect Detection:** Uncovers integration conflicts, syntax bugs, and broken builds immediately after commits.

- **Enhanced Collaboration:** Keeps all engineering team members aligned on a synchronized code state.
- **Reduced Time-to-Market:** Eliminates manual build procedures and accelerates delivery cycles.

Jenkins Automation Server

Jenkins is an open-source, extensible Java-based automation server that serves as the central engine for building, testing, and deploying software projects in Continuous Integration and Continuous Delivery (CI/CD) pipelines.

Jenkins operates by executing jobs across individual agents or worker nodes. Through an extensive ecosystem of plugins, Jenkins connects seamlessly with source control systems (Git, GitHub, GitLab), build systems (Maven, Gradle, npm), testing frameworks, and container engines (Docker, Kubernetes).

Understanding Jenkins Pipelines

A **Jenkins Pipeline** is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins. It defines an automated, repeatable workflow that moves software from raw source code to built artifacts or production deployments.

- **Pipeline as Code:** Pipelines are defined via script files (Jenkinsfile) stored directly alongside application code in version control, enabling version auditing and team tracking.
- **Workflow Visibility:** Breaks down build jobs into clear, observable stages with precise step logs and execution timelines.
- **Complex Flow Support:** Enables parallel execution, conditional stage triggers, retry mechanisms, and automated error recovery.

Declarative vs. Scripted Pipeline Syntaxes

Jenkins supports two distinct scripting syntaxes for orchestrating CI/CD workflows:

Comparison Metric	Declarative Pipeline	Scripted Pipeline
Primary Block	pipeline {}	node {}
Syntax Type	Structured, domain-specific format	Programmatic Groovy code
Learning Curve	Gentle, optimized for readability	Steeper, requires Groovy proficiency

Flexibility	Highly structured with pre-defined rules	Unconstrained, full programmatic control
Error Handling	Native syntax validation in UI	Manual try/catch block structures

2. Pipeline Code Sections

Section A: Declarative Pipeline Code (MY_TEST_1)

The Declarative syntax utilizes strict top-level structural definitions containing agent, stages, stage, and steps keywords.

```
Groovy
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                echo 'Hello, Jenkins!'
            }
        }
    }
}
```

- **pipeline:** Root directive initializing the Declarative configuration block.
- **agent any:** Allocates any available Jenkins executor/node to execute the pipeline steps.
- **stages:** Encloses a sequence of one or more distinct execution phases.
- **stage('Hello'):** Groups logical execution steps under a named visual block in the Jenkins UI.
- **steps:** Defines the actual tasks or shell/echo execution commands to be run within the stage.

Section B: Scripted Pipeline Code (pipeline2)

The Scripted syntax relies on standard Groovy code structures wrapped within worker node blocks.

```
Groovy
node {
    stage('Hello') {
        echo 'Hello, Scripted Pipeline!'
```

```
    }
}
```

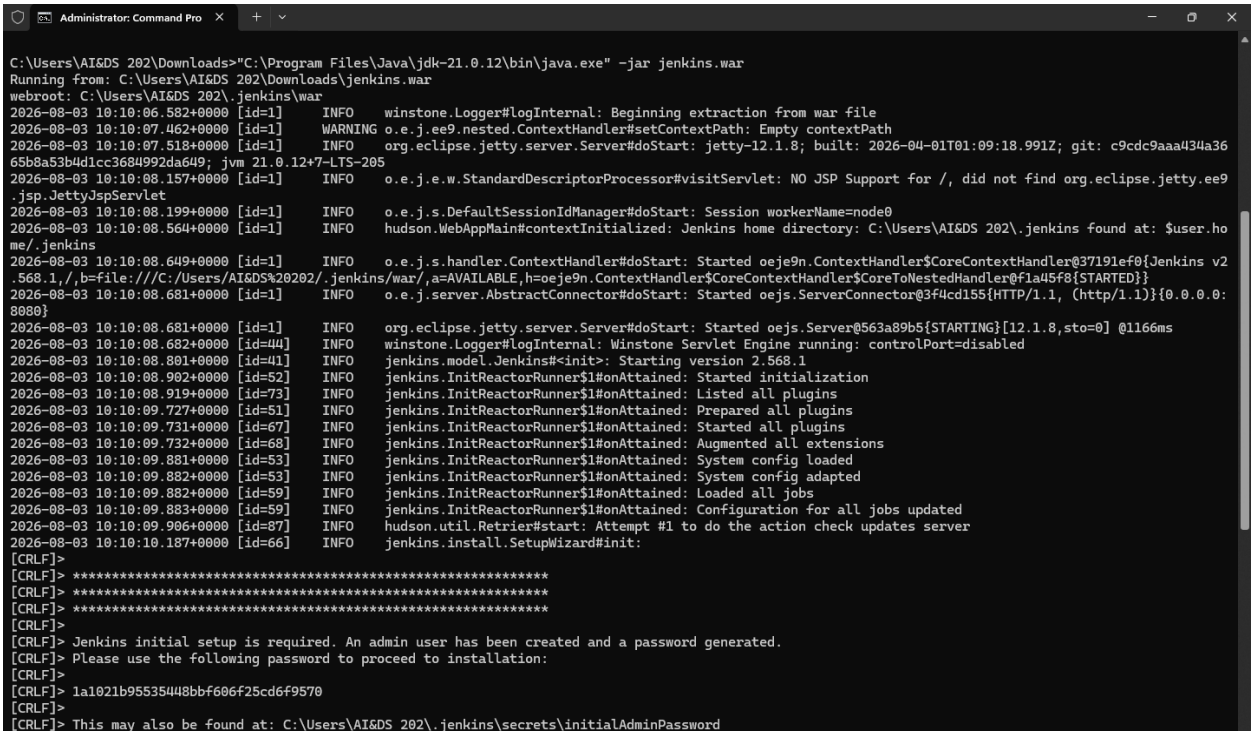
- **node:** Allocates a Jenkins worker instance and workspace directory to execute the enclosed Groovy script block.
- **stage('Hello'):** Defines a stage boundaries marker for step organization and performance logging.
- **echo:** Executes a standard output logging directive directly to the build console output.

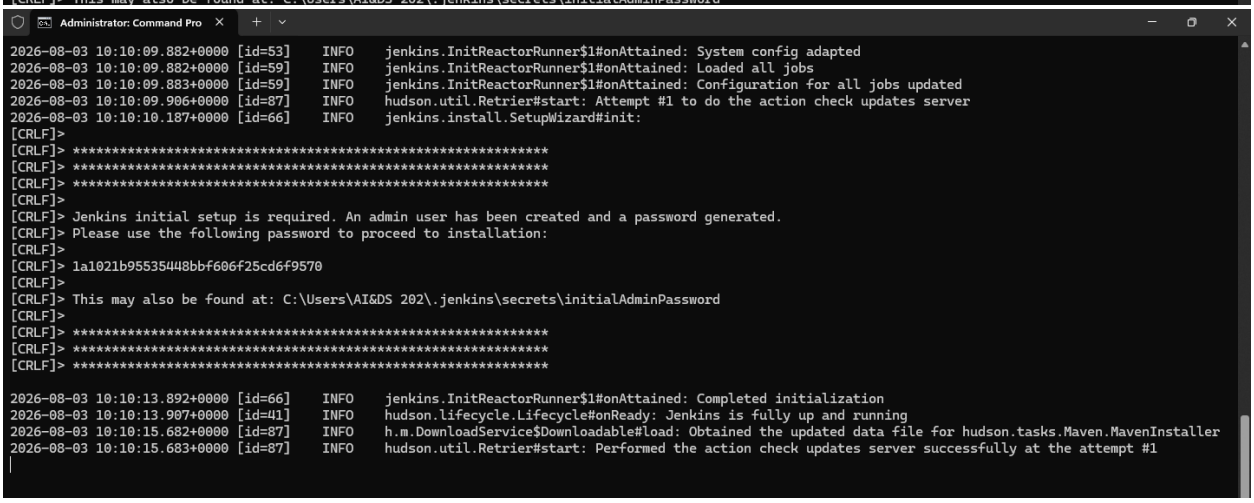
3. Step-by-Step Implementation & Execution

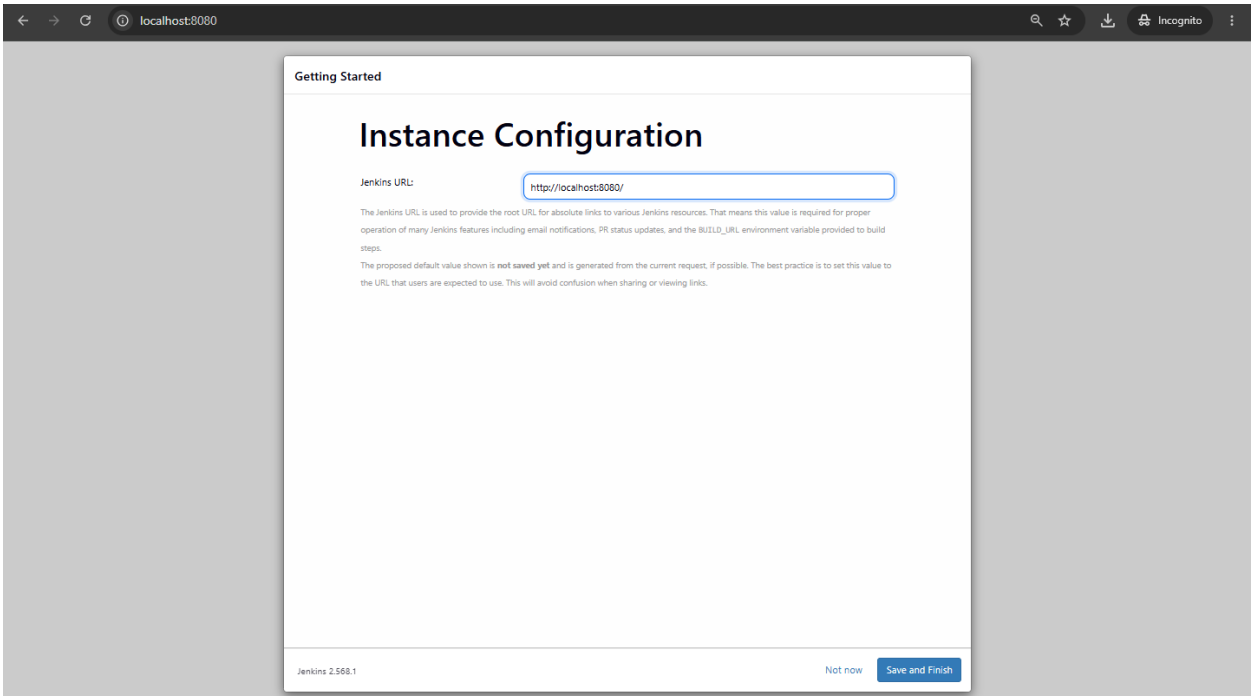
1. **Environment Setup & Server Boot:**
    - Installed Java Development Kit (JDK 21) on the host environment.
    - Executed the generic jenkins.war archive via Command Prompt using the explicit Java binary path:
    - DOS : "C:\Program Files\Java\jdk-21.0.12\bin\java.exe" -jar jenkins.war
    - Monitored initial setup output and retrieved the administrative initial password string generated at C:\Users\AI&DS 202\.jenkins\secrets\initialAdminPassword.
2. **Jenkins Web Setup & Configuration:**
    - Navigated to http://localhost:8080 in the browser and entered the administrative unlock password.
    - Selected **Install suggested plugins** to configure essential core capabilities.
    - Configured administrative credentials and verified instance URL parameters (http://localhost:8080/).
3. **Declarative Pipeline Creation (MY_TEST_1):**
    - Selected **New Item** on the Jenkins dashboard, entered MY_TEST_1, chose **Pipeline**, and confirmed.
    - Configured the **Pipeline script** block with the Declarative code structure and saved.
    - Triggered execution via **Build Now** and confirmed successful output rendering (Hello, Jenkins!) inside **Console Output**.
4. **Scripted Pipeline Creation (pipeline2):**
    - Created a secondary pipeline item named pipeline2.
    - Injected the Scripted Groovy code snippet inside the **Pipeline script** section and saved.
    - Triggered **Build Now** and verified execution log rendering (Hello, Scripted Pipeline!) and Finished: SUCCESS status in the build console.

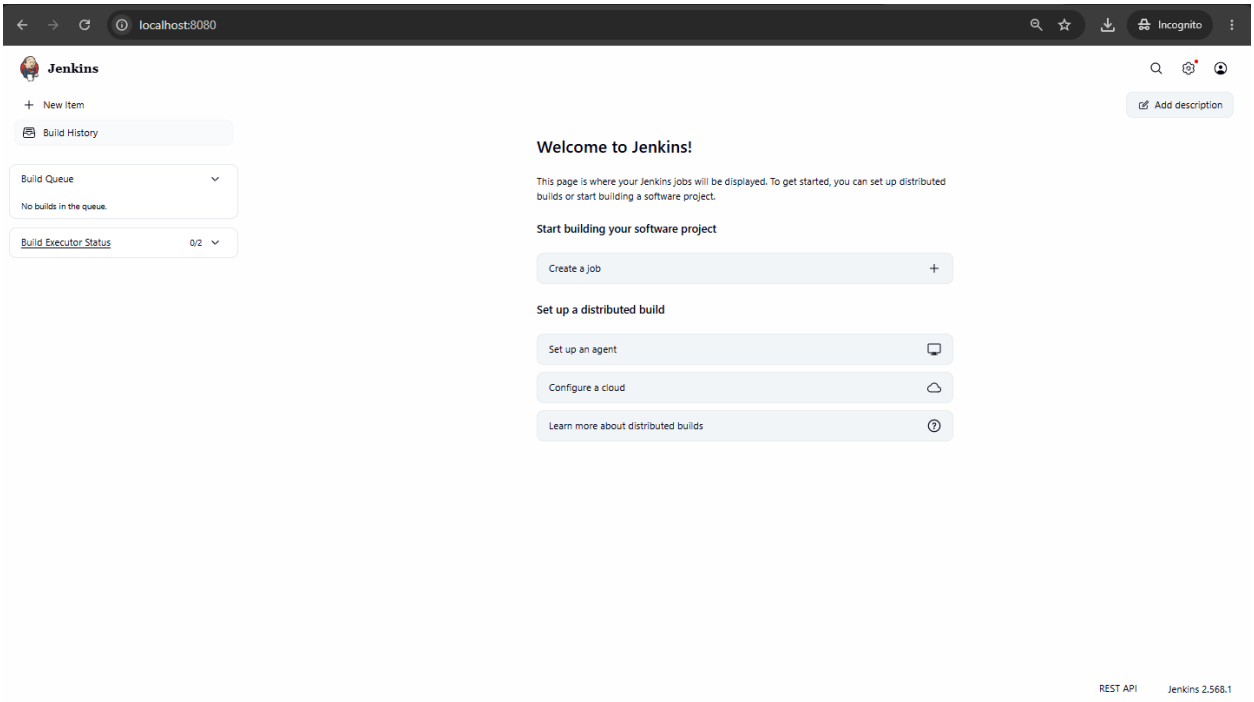
Execution Steps & Screenshot Placement

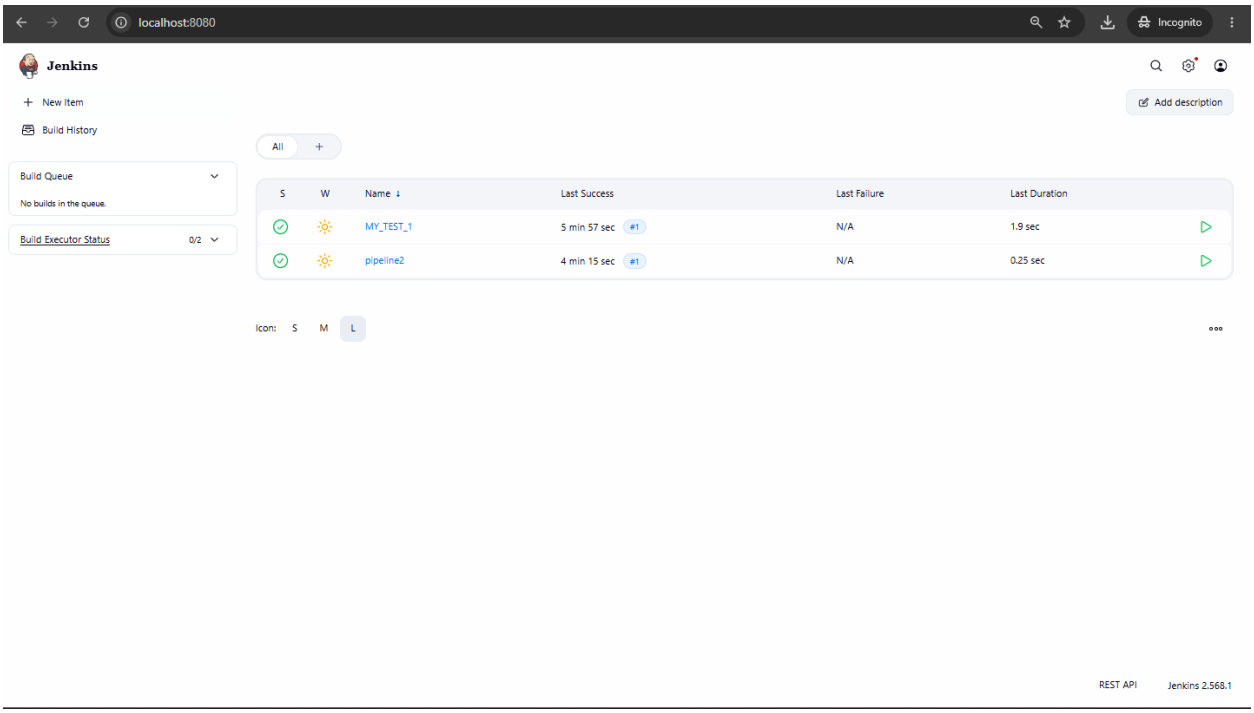
1. **Jenkins Server Setup:** Started Jenkins WAR server using Java 21, completed setup wizard, installed suggested plugins, and configured instance URL.



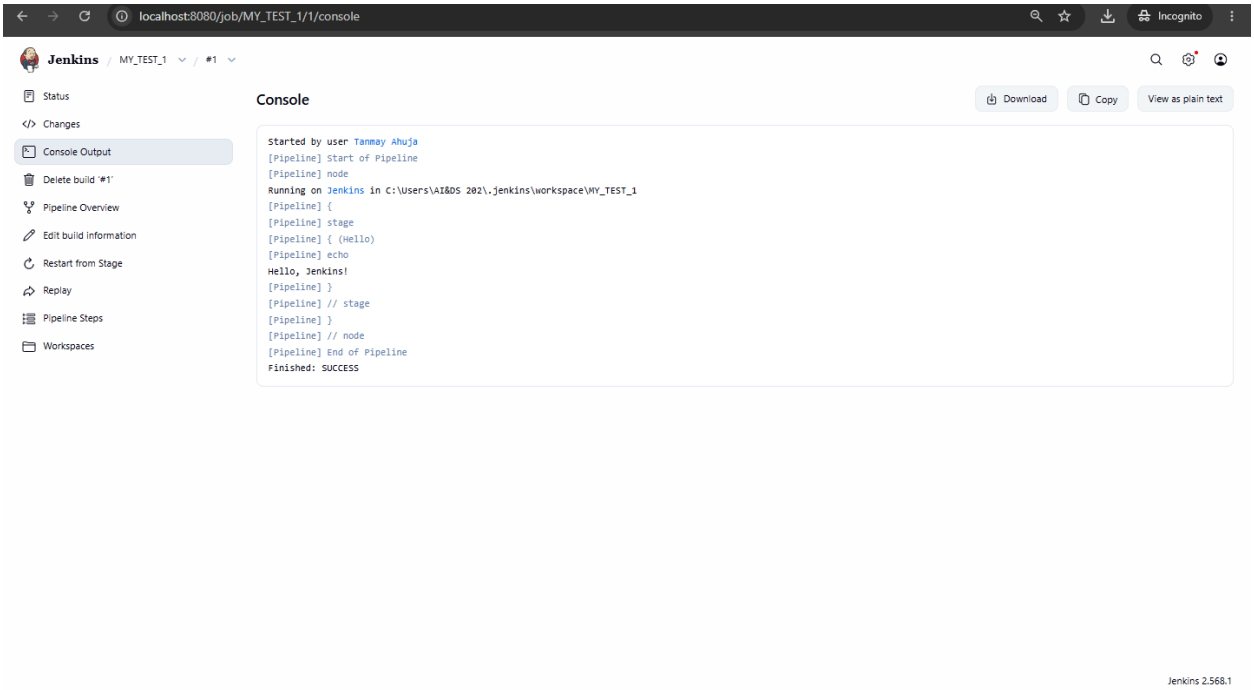



localhost:8080
Incognito
Getting Started
Instance Configuration
Jenkins URL:
http://localhost:8080/
The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.
The proposed default value shown is not saved yet and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.
Jenkins 2.568.1
Not now
Save and Finish

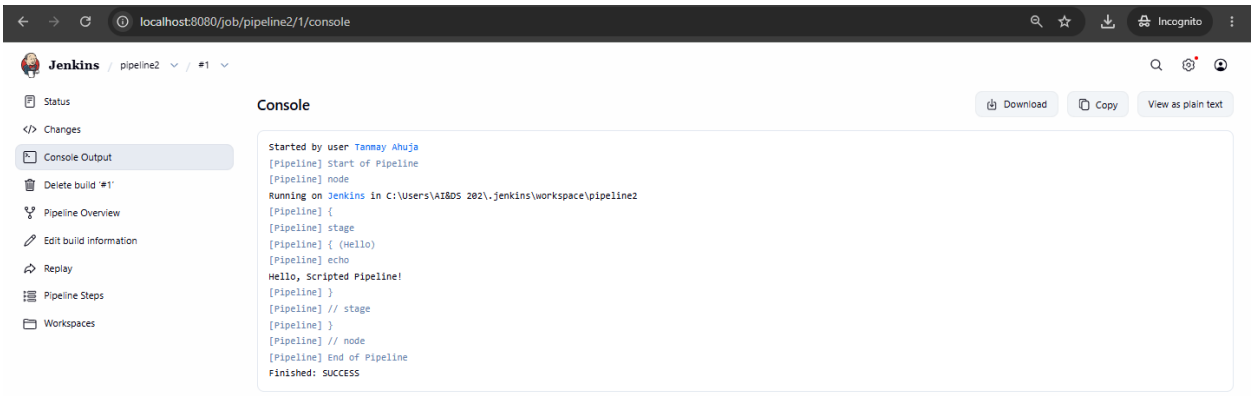

localhost:8080
Incognito
Jenkins
New Item
Build History
Build Queue
No builds in the queue.
Build Executor Status
0/2
Add description
Welcome to Jenkins!
This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.
Start building your software project
Create a job
Set up a distributed build
Set up an agent
Configure a cloud
Learn more about distributed builds
REST API
Jenkins 2.568.1



2. **Declarative Pipeline Creation (MY_TEST_1):** Created a Pipeline item with a pipeline {} block printing 'Hello, Jenkins!'.



3. **Scripted Pipeline Creation (pipeline2):** Created a Pipeline item with a node {} block printing 'Hello, Scripted Pipeline!'.



Conclusion

In this experiment, the fundamental concepts of Continuous Integration (CI) and modern software automation were thoroughly explored and implemented through the deployment of Jenkins on Java 21. By completing the initial web administration setup and plugin initialization, an enterprise-grade automation environment was successfully established to streamline software build workflows. The practical execution involved authoring both Declarative and Scripted pipelines to execute automated build logic using pipeline and node block syntaxes, respectively. Through the creation of the MY_TEST_1 job, the structured, highly readable Declarative Pipeline format was validated via successful console execution. Subsequently, the pipeline2 job demonstrated the versatility of the Groovy-based Scripted Pipeline syntax for running flexible, code-centric build steps. Interactive web console verification confirmed successful build execution and status logging across all pipeline jobs.

Mapping to Learning Outcome 3 (LO3)

This experiment successfully **LO3** (*Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment*). By installing and configuring a Jenkins automation server, authoring pipeline-as-code manifests using both Declarative and Scripted syntaxes, and executing automated build verification inside isolated job workspaces, practical competence in establishing repeatable CI/CD workflows that reduce manual build overhead and accelerate software delivery was effectively demonstrated.